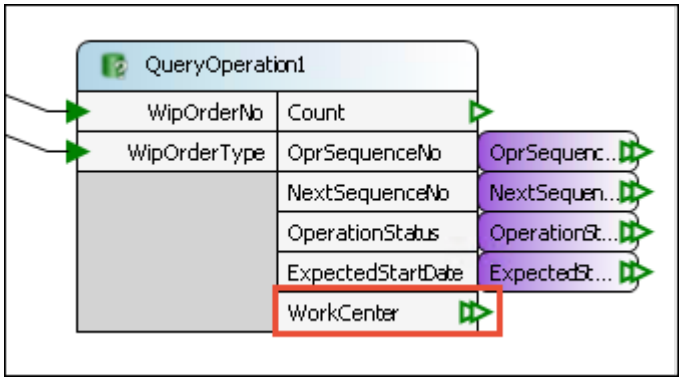
Usage Examples

Extend API

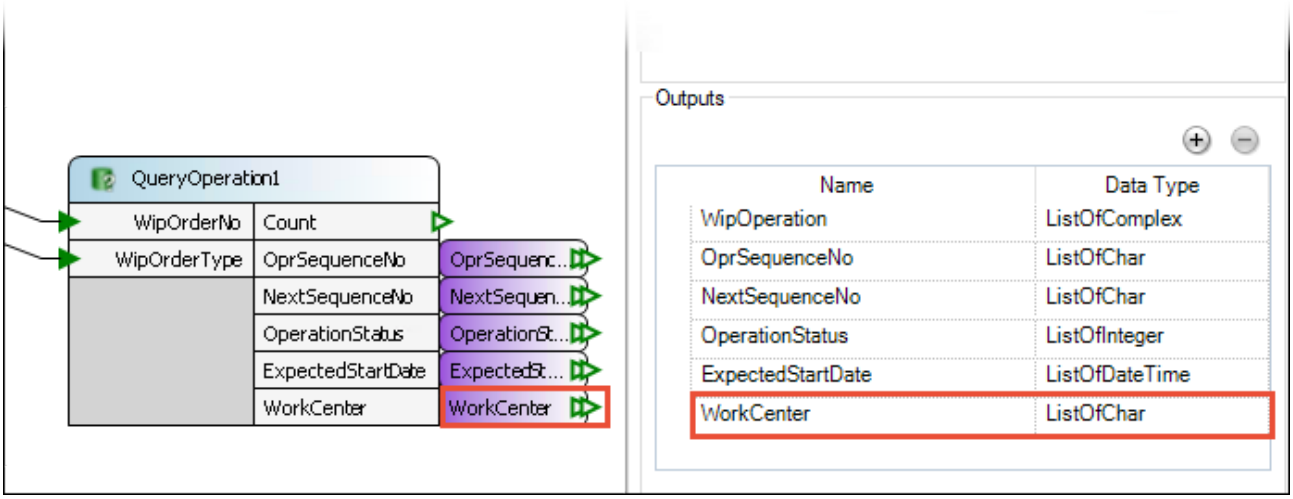
You work with a DFC that queries data. You have already used the data returned by this DFC in several places. The project is growing and you need to query additional data and pass it to different places.

Steps to follow when not using a Complex type

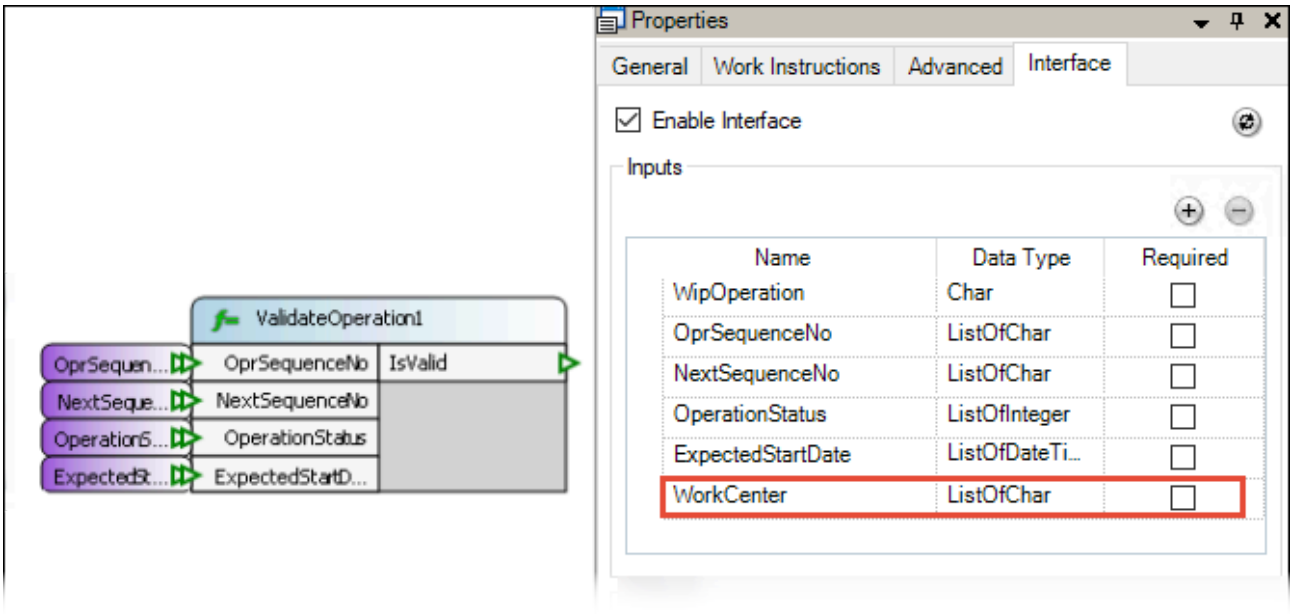
1. Query data from database.



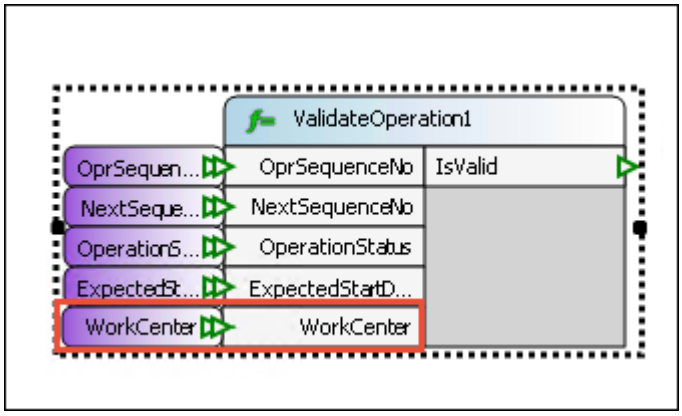
2. Change DFC Interface to return a new variable.



3. Change the external Inputs in the View DFC Interface.
4. Use data to display control editor.
5. Change the external Outputs in the View DFC Interface.
6. Change validation DFC Interface.



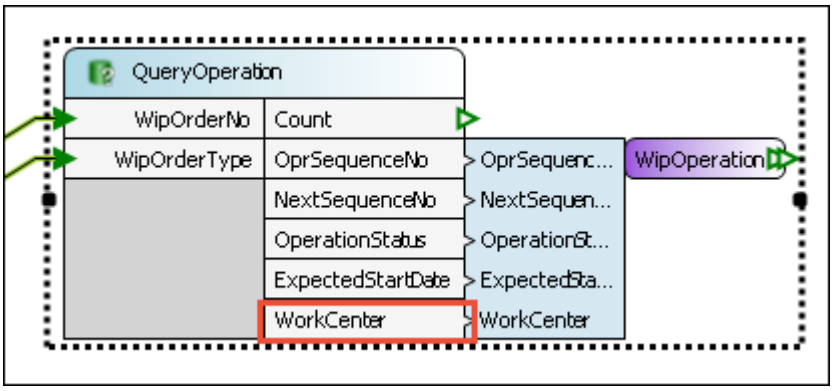
7. Validate edited data.



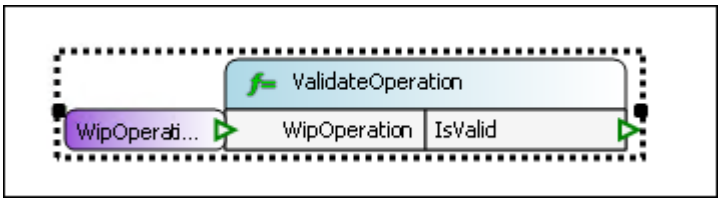
8. Change updating DFC Interface.
9. Update data in database.

Steps to follow when using a Complex type

1. Query data from database.



2. Use data to display control editor.
3. Validate edited data (there is no need to change the Function Inputs)



4. Update data in database.
5. Starting Point

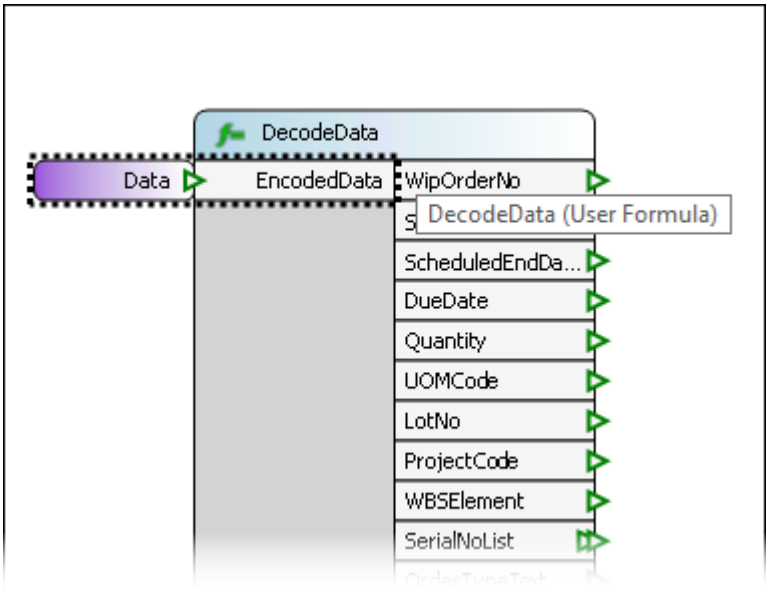
When using a Complex type, you do not need to make any changes to the DFC interface, because it already has all the necessary external variables defined. You only need to implement the business logic.

Process Data From External System

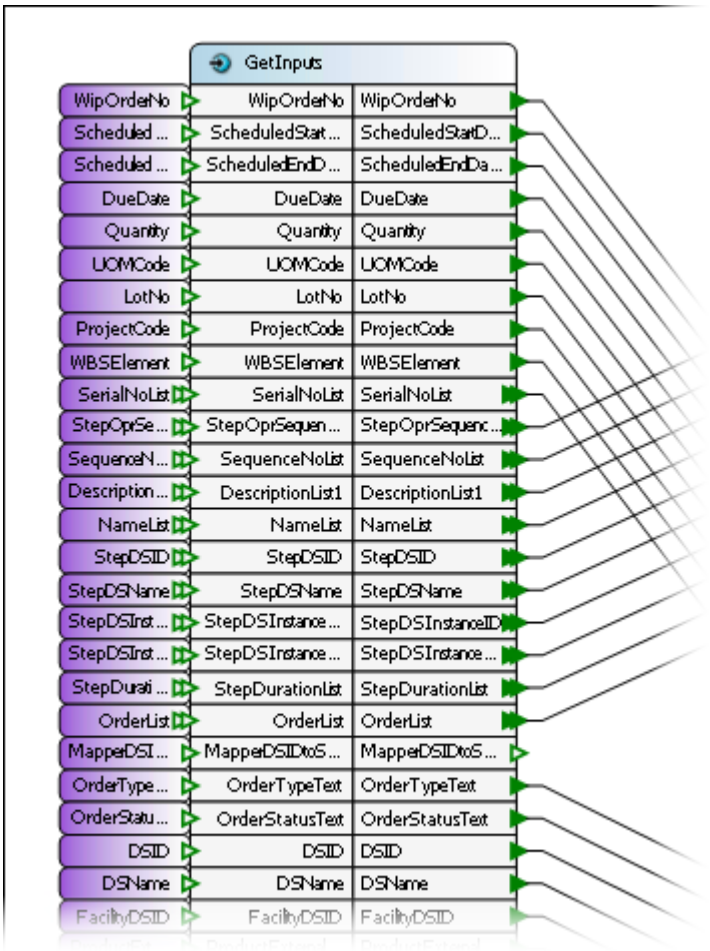
You have to establish a REST HTTP service that can accept large data payload. To do this you need a DFC that can process data in chunks, with separate steps or sub-DFCs that can handle certain parts of data.

Without Complex Type

Without using a Complex type executing such a scenario requires many custom-made User Formulas for selecting and parsing appropriate data chunks. This makes the whole process time consuming and error-prone.



Encoded data that need to be additionally parsed



A large number of external Inputs with simple data types

With Complex Type

When using a Complex Type creating the DFC that can split data to appropriate chunks is easier and much more effective.